A PROJECT REPORT ON
STEALTHLEDGER

SUBMITTED TO
MIT SCHOOL OF COMPUTING, LONI, PUNE IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE

BACHELOR OF TECHNOLOGY
(Computer Science & Engineering)

BY

Vedant Barhate	Enrollment No: ADT23SOCB1288
Vibhor Dev	Enrollment No: ADT23SOCB1303
Achyut Yesare	Enrollment No: ADT23SOCB0041
Yash Pandey	Enrollment No: ADT23SOCB1337

Under the guidance of

Prof. Pratiksha Malvathkar



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

MIT School OF COMPUTING
MIT Art, Design and Technology University

Rajbaug Campus, Loni-Kalbhor, Pune 412201

2024-25



MIT SCHOOL OF COMPUTING
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
MIT ART, DESIGN AND TECHNOLOGY UNIVERSITY,
RAJBAUG CAMPUS, LONI-KALBHOR, PUNE 412201

CERTIFICATE

This is to certify that the project report entitled

“STEALTHLEDGER”

Submitted by

Vedant Nitin Barhate	Enrollment No: ADT23SOCB1288
Vibhor Dev	Enrollment No: ADT23SOCB1303
Achyut Yesare	Enrollment No: ADT23SOCB0041
Yash Pandey	Enrollment No: ADT23SOCB1337

is a bonafide work carried out by them under the supervision of Prof./Dr/ Pratiksha Malvathkar and it is submitted towards the partial fulfillment of the requirement of MIT ADT university, Pune for the award of the degree of Bachelor of Technology (Computer Science and Engineering)

Prof. Pratiksha Malvathkar
Guide

Dr. Ganesh Pathak
Head of Department

Dr. ABC
Director

Dr. DEF
Dean

Seal/Stamp of the College
Place: PUNE

DECLARATION

We, the team members

Name	Enrollment No
Vedant Barhate	(ADT23SOCB1288)
Achyut Yesare	(ADT23SOCB0041)
Vibhor Dev	(ADT23SOCB1303)
Yash Pandey	(ADT23SOCB1337)

Hereby declare that the project work incorporated in the present project entitled **"StealthLedger"** is original work. This work (in part or in full) has not been submitted to any University for the award or a Degree or a Diploma. We have properly acknowledged the material collected from secondary sources wherever required. We solely own the responsibility for the originality of the entire content.

Date: __/11/2024

Name & Signature of the Team Members

Member 1: _____

Member 2: _____

Member 3: _____

Member 4: _____

Name and Signature of Guide

Seal/Stamp of the College

Place: Pune

Date: __/11/2024



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
MIT SCHOOL OF COMPUTING,
RAJBAUG, LONI KALBHOR,
PUNE – 412201

EXAMINER'S APPROVAL CERTIFICATE

The project report entitled "STEALTHLEDGER" submitted by Vedant Barhate (ADT23SOCB1288), Vibhor Dev (ADT23SOCB1303), Achyut Yesare (ADT23SOCB0041), Yash Pandey (ADT23SOCB1337) in partial fulfillment for the award of the degree of Bachelor of Technology (Computer Science & Engineering) during the academic year 2024-25, of MIT-ADT University, MIT School OF COMPUTING, Pune, is hereby approved.

Examiners:

1.

2.

ACKNOWLEDGEMENT

We would like to express my sincere gratitude to all those who have contributed to the successful completion of this project, StealthLedger: A Blockchain-Based Steganographic Data Storage System.

First and foremost, we are deeply thankful to our project guide, **Prof. Pratiksha Malvathkar**, for their invaluable guidance, constructive feedback, and constant encouragement throughout the development of this project. Their expertise and support have been instrumental in shaping this work.

We are grateful to the faculty members and staff of **MIT SOC** who provided the resources and a conducive environment for carrying out this research. Their insightful lectures and discussions served as a foundation for exploring this innovative combination of blockchain and steganography.

We extend my heartfelt thanks to my peers and colleagues for their critical inputs, engaging discussions, and moral support during the challenges encountered in this project. Their perspectives helped refine and improve the system.

We would also like to acknowledge the online open-source community and the developers of various tools and libraries, such as Python's **bcrypt**, **SQLite** etc. and the **Steghide** tool, whose work made it possible to implement and test key functionalities efficiently.

Lastly, we are profoundly thankful to our family and friends for their unwavering support, patience, and motivation throughout this journey. Their belief in our abilities inspired us to persevere and deliver this project to the best of my capabilities.

This project is the result of collaborative efforts, and I sincerely appreciate the contributions of everyone involved.

Name, Enrollment No.
Vedant Barhate, ADT23SOCB1288
Vibhor Dev, ADT23SOCB1303
Achyut Yesare, ADT23SOCB0041
Yash Pandey, ADT23SOCB1337

ABSTRACT

This paper introduces StealthLedger, a novel approach to secure data storage that combines blockchain technology with image steganography. In an era where data breaches and privacy concerns are increasingly prevalent, we present a system that enables users to hide sensitive information within images while leveraging blockchain's immutable nature for secure storage. Unlike traditional blockchain implementations, StealthLedger uses a non-distributed architecture, providing each user with their own private blockchain accessible through a unique API token.

Our work demonstrates how image steganography, specifically the LSB (Least Significant Bit) technique through the Steghide tool, can be effectively integrated with blockchain technology to create a robust security solution. The system employs bcrypt for passcode hashing and implements a graph-theoretic approach to data embedding, ensuring both the confidentiality of hidden data and the integrity of the storage mechanism.

The system proved particularly effective in scenarios requiring secure document sharing, confidential communication, and digital watermarking. While current limitations include blockchain storage scalability and computational overhead during data embedding, these challenges present opportunities for future optimization.

This research contributes to the growing field of privacy-preserving technologies by demonstrating how traditional cryptographic techniques can be enhanced through blockchain integration. The practical applications of StealthLedger span various domains, from corporate data security to medical record management, offering a versatile solution for organizations seeking to protect sensitive information while maintaining complete control over their data.

CONTENTS

Certificate ……...……. i

Declaration ……..……..……...……. ii

Examiner's Approval Certificate ……. iii

Acknowledgement ……..……...……. iv

Abstract ……..…...……. v

List of Figures ……..…….……. vii

List of Tables ……..…….……. viii

Chapter 1 INTRODUCTION 1

1. Introduction 1

2. Existing Work 1

3. Problem Statement and Motivation 1

4. Objectives 2

5. Scope 2

Chapter 2 CONCEPTS AND METHODS 3

2.1 Core Concepts 5

2.2 Methodology 4

Chapter 3 LITERATURE SURVEY 17

3.1 Stego-Chain: Blockchain-integrated Image Steganography 17

3.2 PSO-Blockchain for COVID-19 Data Security in Healthcare 17

3.3 Blockchain and Image Steganography for Decentralized Authentication 18

3.4 Blockchain and Steganography based Research 18

Chapter 4 PROJECT PLAN 21

Chapter 5 SOFTWARE REQUIREMENT SPECIFICATION 24

5.1 Introduction 24

5.2 Overall Description 26

5.3 Specific Requirements 27

5.4 Supporting Information 28

Chapter 6 RESULTS 29

Chapter 7 SOFTWARE TESTING 31

7.1 Testing Overview 32

7.2 Key Observations 34

Chapter 8 CONCLUSION AND FUTURE WORK 34

BIBLIOGRAPHY 35

ANNEXURE A: List of Publications and Research Paper (In its Original formats) 36

ANNEXURE B: Plagiarism Report 37

ANNEXURE C: Empathy Chart and Solution Document

Annexure D: Project Tracker

LIST OF FIGURES

Figure Number: Figure of the table	Page Number
FIGURE 1: SOFTWARE MODELING	23

LIST OF TABLES

Table Number: Title of the table	Page Number
TABLE 1: LITERATURE REVIEW	23
TABLE 2: RESULTS TABLE	29

Chapter 1 INTRODUCTION

1. Introduction

StealthLedger is an innovative project that combines blockchain technology with image steganography, creating a secure and private system for storing and retrieving sensitive information. By hiding data within images and storing them on a blockchain, StealthLedger provides an immutable, decentralized method of managing sensitive data. This system is designed with user control in mind, offering each user a private blockchain accessible only via a unique API token.

This report presents the project's technical aspects, including its motivation, design, functionalities, and potential applications.

2. Existing Work

Blockchain and steganography have been studied together in recent research for enhancing various applications' security and data authenticity. Here, one of the major techniques proposed by Sarkar et al is the Stego-chain architecture that leverage blockchain to preserve steganography contents from alteration. This method helps to keep the classified material secure as it is in transfer, and provides an additional capacity and some degree of anti-tampering over the usual steganalysis methods. Another example is the work by Mohsin et al., which proposed a blockchain based steganography system for sharing patients' data during Covid-19 pandemic. Their system utilizes Particle Swarm Optimization (PSO) algorithm in order to enhance usage of image pixels and security and accuracy of data in the decentralized healthcare domain networks. Further, Jahanvi et al developed an architecture for dispensed affirmation with the help of blockchain and steganography, they used the Ethereum platform and placed user identity into images and converted them into NFT. This makes it possible to confidently check the identity of individuals without having to rely on the central structures. The combination of blockchain and steganography provides an opportunity for increasing the confidentiality, object integrity, and decentralization, as shown by the results of these works. The StealthLedger project enhances the applicability of these ideas by providing the data to be stored on a non-disclosed blockchain architecture, the main issue of the existing approaches to this problem, including payload capability, as well as access control.

3. Problem Statement and Motivation

With the rising threats in the cyber world, conventional approaches to data storage and authentication comes with added risks of tampering, breaches, or single points of failure. Traditional centralized systems make data vulnerable to unauthorized users since the data transit over insecure networks.

These challenges are what StealthLedger is driven by and presents a solution that improves security through the use of blockchain and steganography. Its purpose is to offer a non-distributed immutable ledger technology that improves the privacy of users and data integrity in transmission.

4. Objectives

The main objectives of StealthLedger are:

- **Data Hiding with Image Steganography**: Embed user data within images using steganography, specifically the Least Significant Bit (LSB) technique.
- **Blockchain Storage for Data Integrity**: Utilize blockchain technology to secure, store, and maintain the immutability of steged images.
- **User-Specific Private Blockchains**: Create an isolated, private non-distributed blockchain for each user, ensuring personalized data security.
- **API-Based Access Control**: Allow users to interact with the system through a web API, which will incorporate token-based authentication in future iterations.

5. Scope

The scope of the StealthLedger project is to develop a secure, private system for storing and retrieving sensitive data by combining blockchain technology with image steganography. The system enables users to embed confidential data within image files, ensuring privacy and data integrity, and storing it in an immutable blockchain structure. Unlike traditional blockchain implementations, StealthLedger provides each user with a private, non-distributed blockchain accessible only through a unique API token, making it highly customizable for individual security needs. This project targets applications in secure document sharing, confidential communication, and digital watermarking, aiming to offer a robust solution for sectors requiring high standards of data protection, such as corporate, healthcare, and legal domains. Future enhancements will focus on expanding multimedia support and integrating API-based access for broader accessibility and

usability

Chapter 2 CONCEPTS AND METHODS

2.1 Core Concepts:

1. **Steganography:**
    - **Definition:** The art of hiding data within a medium, such as images, to make it undetectable.
    - **Technique Used:** Least Significant Bit (LSB) method via **Steghide**.
        - Embeds sensitive data into the least significant bits of image pixels, ensuring minimal visual distortion.
        - **Graph-Theoretic Approach:** Selects specific pixel locations for embedding, enhancing security.
2. **Blockchain Technology:**
    - **Definition:** A decentralized ledger that ensures data integrity and immutability through cryptographic hashes.
    - **Key Features:**
        - Each block contains an index, timestamp, data, and hash of the previous block.
        - **Non-Distributed Blockchain:** Unique to each user, ensuring personalized security.
3. **Cryptography:**

- **Encryption Method:** AES-128 in CBC mode secures sensitive data before embedding.
- **Passcode Hashing:** Utilizes **bcrypt** for robust passcode protection against brute-force attacks.

2.2 Methodology

2.2.1 Steganography Process

The steganography module of StealthLedger utilizes image-based data hiding through the Least Significant Bit (LSB) method, which conceals data in the least significant bits of an image's pixel values. This technique ensures that the hidden information remains difficult to detect without the appropriate decryption or extraction algorithm, making it an effective method for embedding sensitive data within digital images.

In this project, the tool Steghide is used to perform the image-based steganography. Steghide embeds data into the Least Significant Bits (LSB) of an image's pixel values, making alterations to the image undetectable to the naked eye. This method is widely adopted because changes to the least significant bits generally have minimal impact on the visual appearance of the image. Additionally, Steghide provides encryption features and robust handling of hidden data.

The encryption process in Steghide begins when the user provides a passcode, which is used both to encrypt the data and control the embedding process. The data to be hidden is first compressed to reduce its size. Steghide then applies the AES-128 encryption algorithm in CBC mode to encrypt the compressed data, using the entered password. After encryption, Steghide employs a graph-theoretic approach (rather than a simple LSB method) to embed the data into specific locations in the cover image file. This ensures that the data does not interfere with the image's visual integrity, and the hidden information remains secure.

Steganography Process Flow:

1. Hiding Data in an Image: The `SteganographyTool` class encapsulates the process of hiding and extracting data from an image using Steghide. The class first ensures that the

provided image format is supported (e.g., JPEG, PNG, BMP). It then embeds the data in the image as follows:

- Data Preparation: The data to be hidden is serialized using Python's `pickle` module and stored in a temporary file.
- Image Saving: The input image (in binary form) is loaded and saved into a temporary directory to prepare it for data embedding.
- Data Embedding with Steghide: The `steghide` command-line tool is invoked using a subprocess to embed the hidden data into the image. The following command structure is used:

```bash

steghide embed -cf <input_image> -ef <data_file> -sf <output_image> -p <passcode>

```

- `-cf`: Cover file (input image).
- `-ef`: Data file (the serialized data).
- `-sf`: Stego file (output image).
- `-p`: Password used for encryption and data embedding.

The result is an output image (`output_image`) that contains the hidden data, which can later be extracted using the correct passcode.

```python

def hide_data_in_image(self, data, input_image_data, passcode):

    with tempfile.TemporaryDirectory() as temp_dir:

        temp_input_image_path = os.path.join(temp_dir, f"input_image.{self.file_format}")

        temp_output_image_path = os.path.join(temp_dir, f"output_image.{self.file_format}")

        temp_data_path = os.path.join(temp_dir, "temp.bin")
```

```
        with open(temp_data_path, "wb") as temp_file:
            pickle.dump(data, temp_file)
        ...
```

2. Extracting Hidden Data from an Image: Once the data is embedded into an image, it can be extracted using the same passcode used for embedding. The `extract_data_from_image` method is responsible for this process. It works as follows:

- Image Preparation: The steged image (the image with hidden data) is saved into a temporary file to be processed by Steghide.
- Data Extraction with Steghide: The `steghide` tool is invoked using the following command structure to extract the hidden data:

  ```bash
  steghide extract -sf <steged_image> -xf <output_file> -p <passcode>
  ```

  - `-sf`: Steged file (the image with embedded data).
  - `-xf`: Output file where the extracted data will be saved.
  - `-p`: Password used for extraction.
- Data Retrieval: After Steghide completes the extraction, the hidden data is read from the output file, deserialized using `pickle`, and returned to the user.

  ```python
  def extract_data_from_image(self, steg_image_data, passcode):
      with tempfile.TemporaryDirectory() as temp_dir:
          temp_steg_image_path = os.path.join(temp_dir, f"steg_image.{self.file_format}")
```

```
    outfile_name = os.path.join(temp_dir, "out_bin.bin")

    save_image_to_temp(steg_image_data, temp_steg_image_path)

    ...
```

Workflow of Data Hiding and Extraction:

Hiding Data:

1. Prepare Data: The data to be hidden is serialized into a binary format using `pickle`.

2. Store Data Temporarily: The serialized data is saved in a temporary file.

3. Save Image Temporarily: The input image (in binary format) is saved into a temporary file.

4. Embed Data with Steghide: The `steghide` tool is used to embed the hidden data into the image, using the password for encryption and embedding. The data is not simply appended to the image; it is carefully embedded into specific pixel locations to avoid visual distortion.

5. Return Steged Image: The resulting image (with hidden data) is returned.

Extracting Data:

1. Load Steged Image: The steged image (with hidden data) is loaded into a temporary file.

2. Extract Data with Steghide: The `steghide` tool is used to extract the hidden data from the image. The password provided during the embedding process is used to decrypt and extract the data.

3. Retrieve Data: The extracted binary data is deserialized using `pickle`, and the original hidden data is returned.

Steghide Tool and Graph-Theoretic Approach:

Steghide is an open-source tool that uses the Least Significant Bit (LSB) method to hide data in cover files like images. However, instead of simply overwriting the least significant bits of every pixel, Steghide uses a graph-theoretic approach (which is like advanced LSB method) to select specific pixel locations within the cover image for embedding the hidden data. This careful selection helps ensure that the embedded data does not visually distort the image. The result is a more secure and undetectable method for hiding information.

Security and Integrity:

The security of the data hiding process is ensured by AES-128 encryption applied to the hidden data before embedding it in the image. The encryption is controlled by a passcode, ensuring that only users with the correct passcode can extract and decrypt the hidden data. The use of the graph-theoretic method further enhances the security of the data hiding process by embedding the data in specific locations within the image, rather than uniformly across all pixel values.

2.2.2 Blockchain Technology

Blockchain Structure:

The blockchain system implemented in this project follows a simple yet functional approach to storing and verifying data in a non-distributed manner. The blockchain is composed of a series of blocks, each containing data and a cryptographic hash that links it to the previous block, ensuring data integrity and immutability. The design is inspired by foundational blockchain principles but adapted for local storage using an SQLite database and Python libraries.

Key Components of the Blockchain System:

The primary components of the blockchain in this project are the Block class and the Blockchain class.

1. Block Class:

A block is the basic unit of the blockchain. Each block contains the following attributes:

- Index: The position of the block in the chain (starting from 0 for the genesis block).
- Data: The payload or content stored within the block. In this case, the data is serializable object of an image file with hidden data.
- Previous Hash: The cryptographic hash of the previous block, which establishes the chain's continuity and immutability.
- Timestamp: The time when the block was created.
- Hash: The unique SHA-256 hash of the block, which is computed based on the block's contents.

The `Block` class in the code performs the following operations:

- `calculate_hash()`: This method computes a SHA-256 hash for the block by serializing the block's attributes (index, data, previous hash, and timestamp) into a string format and then encoding it for hashing. The resulting hash is stored as the block's unique identifier.

```python
def calculate_hash(self):
    block_data = {
        'index': self.index,
        'data': str(self.data),
        'previous_hash': self.previous_hash,
        'timestamp': self.timestamp
    }
```

```
block_string = ujson.dumps(block_data, sort_keys=True).encode()

return hashlib.sha256(block_string).hexdigest()
```

2. Blockchain Class:

The Blockchain class manages the collection of blocks and ensures the integrity and validation of the blockchain. The blockchain's functionality is implemented as follows:

- Chain Storage: The blockchain is stored in a list (`self.chain`), where each block is appended after being validated.
- Database Integration: An SQLite database (`database.db`) is used to persist the blocks. This ensures that the blockchain can be loaded and restored between application restarts. The database stores blocks as serialized objects (using Python's `pickle` module) along with an associated identifier and passcode for each entry.

```python
self.conn = sqlite3.connect('database.db')
```

Block Addition: When a new block is added to the chain, the `add_block()` method:

- Calculates the block's index (based on the index of the latest block).
- Determines the previous block's hash to link the blocks together.
- Serializes the block and stores it in the SQLite database.
- Appends the new block to the blockchain's internal list.

```python
def add_block(self, data, identifier, passcode):
    latest_block = self.get_latest_block()
    index = latest_block.index + 1 if latest_block else 0
    previous_hash = latest_block.hash if latest_block else "0"
    new_block = Block(index=index, data=data, previous_hash=previous_hash)
    self.chain.append(new_block)
```

serialized_block = pickle.dumps(new_block)

with self.conn:

self.conn.execute(f"INSERT INTO {self.name} (block, identifier, passcode) VALUES (?, ?, ?)", (serialized_block, identifier, passcode))

```

Chain Validation: The `is_chain_valid()` method checks the integrity of the blockchain by verifying:

- That the hash of each block matches its calculated hash.
- That the `previous_hash` of each block correctly matches the hash of the previous block.

```python

def is_chain_valid(self):

for i in range(1, len(self.chain)):

current_block = self.chain[i]

previous_block = self.chain[i - 1]

if current_block.hash != current_block.calculate_hash():

return False

if current_block.previous_hash != previous_block.hash:

return False

return True

```

Chain Restoration: On initialization, the blockchain attempts to load existing blocks from the database by deserializing them into `Block` objects and appending them to the chain.
```

```python
def load_chain_from_db(self):
    cursor = self.conn.execute(f"SELECT block FROM {self.name} ORDER BY block_id")
    rows = cursor.fetchall()
    for row in rows:
        block = pickle.loads(row[0])
        self.chain.append(block)
```

Genesis Block: The blockchain begins with a genesis block (the first block), which is created automatically when the blockchain is initialized. This block does not have a predecessor and its data is predefined (in this case, the string "Genesis_Block").

```python
if len(blockchain.chain) == 0:
    blockchain.add_block(b"Genesis_Block", "0th block", "")
```

Database Interaction: Each time a new block is added, the block is serialized using the `pickle` module and stored in an SQLite database. The database ensures the persistence of the blockchain across application restarts. This design allows for a lightweight, persistent storage solution for managing the blockchain's state.

Security and Integrity: The blockchain relies heavily on the `hashing` function to ensure data integrity. Each block's hash is computed based on its contents, and this hash is stored within the block. If any data within a block changes, the hash would be different, and this discrepancy would immediately be detectable when validating the chain. Additionally,

since each block contains a reference to the previous block's hash, any tampering with a block would invalidate the entire chain, making it impossible to alter the data without detection.

Blockchain Integrity Check: To ensure the blockchain's validity, the `is_chain_valid()` method checks the consistency of the entire chain. It compares the hash of each block with its calculated hash and verifies that the `previous_hash` of each block matches the hash of its predecessor. If any inconsistencies are found, the blockchain is considered invalid.

2.2.3 Creating StealthLedger – Integrating Blockchain and Steganography

The StealthLedger is a secure data storage system that integrates **blockchain** technology and **steganography** to provide a robust solution for hiding and retrieving sensitive data in a non-distributed and secure manner. The system combines the immutability of blockchain with the confidentiality of image-based data hiding, making it possible to store hidden data within images while ensuring that the data remains securely tied to the blockchain.

The key idea behind `StealthLedger` is to use blockchain for verifying the integrity of the stored data and steganography to conceal this data within images. The blockchain serves as the underlying structure for storing encrypted and hidden data, while steganography ensures that the data is hidden in a way that is difficult to detect without the correct passcode.

Key Components:

The StealthLedger system consists of two main components:

1. Blockchain Module: This component handles the creation, storage, and validation of blocks in the blockchain.

2. Steganography Module: This component is responsible for hiding data within images and extracting it later.

Additionally, bcrypt is used to securely hash the passcodes used for encrypting and retrieving hidden data, ensuring that the passcodes are stored securely in the blockchain.

StealthLedger Class Design:

The StealthLedger class serves as the main interface for interacting with the system. It integrates both the blockchain and steganography components and provides methods for adding new blocks and retrieving data from the blockchain. Here's an in-depth look at the functionality:

1. Initialization: The `StealthLedger` class is initialized with a name and file format. Upon initialization, it creates an instance of the SteganographyTool (for handling image-based steganography) and the Blockchain (for creating and managing the ledger).

```python
def __init__(self, name, file_format) -> None:
    self.name = name
    self.file_format = file_format
    self.steganography = SteganographyTool(file_format=self.file_format)
    self.blockchain = create_blockchain(self.name)
```

2. Passcode Hashing: To ensure the security of passcodes, the system uses bcrypt to hash the passcodes before storing them in the blockchain. This ensures that the passcodes are not stored in plaintext, adding a layer of security to the system.

```python
def _hash_passcode(self, passcode):
    """Hashes the passcode using bcrypt."""
```

```
    return bcrypt.hashpw(passcode.encode('utf-8'), bcrypt.gensalt())
```

3. Adding a Node: When adding a new block to the blockchain, the system performs the following steps:

- Hide Data: The sensitive data is hidden within an image using the steganography module. The data is encrypted and embedded into the image's least significant bits using Steghide.
- Hash Passcode: The provided passcode is hashed using bcrypt to ensure that the passcode is stored securely.
- Add to Blockchain: The encrypted image (with the hidden data) and the hashed passcode are then added to the blockchain as a new block. This block is serialized and stored in the blockchain.

```python
def add_node(self, data, image, identifier, passcode):
    with open(image, 'rb') as img_file:
        img_data = img_file.read()
            steged_img_data = self.steganography.hide_data_in_image(data, img_data, passcode)
    hashed_passcode = self._hash_passcode(passcode)
    self.blockchain.add_block(steged_img_data, identifier, hashed_passcode)
```

4. Retrieving Data: To retrieve hidden data from the blockchain, the following steps are performed:

- Retrieve Block: The block associated with the given identifier is retrieved from the blockchain.
- Verify Passcode: The passcode entered by the user is checked against the stored hash (using bcrypt). If the passcodes match, the system proceeds to extract the hidden data.
- Extract Hidden Data: The data hidden inside the image is extracted using the SteganographyTool, which extracts and decrypts the data using the same passcode.

```python
def retrive_from_node(self, identifier, passcode):
    with self.blockchain.conn:
            cursor = self.blockchain.conn.execute(f"SELECT block, passcode FROM {self.name} WHERE identifier = ?", (identifier,))
        record = cursor.fetchone()
    if not record:
        return f"No Node with the given identifier: {identifier} found."

    serialized_block, stored_hashed_passcode = record
    # Verify the passcode
    if not bcrypt.checkpw(passcode.encode('utf-8'), stored_hashed_passcode):
        return "Passcode does not match."
    # Deserialize the block and extract data from the image
    block = pickle.loads(serialized_block)
    img_data = block.data
    hidden_data = self.steganography.extract_data_from_image(img_data, passcode)
    return hidden_data
```

Workflow of StealthLedger:

1. <u>Adding Data to the Ledger:</u>

- The user provides data to be hidden, along with an image and a passcode.

- The data is hidden within the image using steganography.
- The passcode is hashed using bcrypt, and the image (containing the hidden data) along with the hashed passcode is stored in the blockchain.

2. Retrieving Data from the Ledger:

- The user provides the identifier and passcode to retrieve hidden data.
- The blockchain is queried for the block associated with the given identifier.
- The passcode is verified against the stored hash. If correct, the hidden data is extracted from the image using steganography.

Chapter 3 LITERATURE SURVEY

The StealthLedger project integrates blockchain with steganography to offer a secure, tamper-proof solution for data storage. This approach is supported by recent literature

exploring similar techniques in various applications. Key studies in this field have investigated innovative blockchain-steganography combinations to address issues such as data integrity, confidentiality, and decentralized access, especially in sensitive domains like healthcare and authentication.

3.1 Stego-Chain: Blockchain-integrated Image Steganography

The “Stego-chain” framework proposed by Sarkar et al. is based on the upgrade of the steganographic data integrity with the help of blockchain. With the help of damaged Robert’s edge detection, this method needs to increase the areas of edges in the picture in order to enhance the steganalysis of the payload capacity of the stego-images transmitted through a peer-to-peer network. Such a design guarantees that secret data residing on images is well protected from tampering because blockchain performs an excellent job in preventing alterations through its inherent features. The functionality shown in the study illustrates that this approach outperforms conventional steganography, both in the payload capability and resistance to tampering in the transmission pathways.

3.2 PSO-Blockchain for COVID-19 Data Security in Healthcare

Due to the pandemic caused by SARS-CoV- 2 virus, Mohsin et al suggested a blockchain based steganography for sharing and updating the patient data on the hospital networks. This system uses Particle Swarm Optimization for data embedding with the main goal of getting the best pixel utilization in stego image while at the same time guaranteeing high image quality and image security. Unlike the other methods, this method draws advantage of the distributed structure of blockchain and it enhance data availability and data integrity when transmitted through the health care networks that are centralized. By such a design, hospitals can exchange such vital medical information in a secure manner, hence overcoming these centralized system drawbacks to levels of confidentiality and reliable data during network failures.

3.3 Blockchain and Image Steganography for Decentralized Authentication

The authors Jahanvi et al. proposed a blockchain-steganography model based on the Ethereum platform to verify the users' identity. In this context, the user identity information is embedded into pictures using steganography and converted into NFTs residing on Ethereum. This approach reduces potential challenges associated with the centralized system of authentication by giving a secure and auditable proof of the identity of users across various networks. The decentralized verification of data is facilitated through use of NFT on a public blockchain in this method. This approach is especially pertinent to StealthLedger, which also aims to create the secure and decentralized data management in the framework of blockchain-steganography amalgamation.

3.4 Blockchain and Steganography based Research

The paper, "Image Steganography: Article summarized "Recent Progress in Steganography: A Review of the Recent Advances" (2021), where the author stresses the developments of new methods to steganography, though the LSB method, and the CNNs and GANs were used. These developments optimise payload capability, stealth, and security of information. This review was focused on the steganography opportunity of embedding dangerous data into images that form a basis for LSB-based steganography in StealthLedger for safe data hiding (Image_Steganography_A_R…).

The paper, "SteganoGAN: This study published in" Efficient High-Capacity Image Steganography with GANs" (2019) proposed an adversarial framework that is known to achieve up to 4.4 bits per pixel to hide digital images and videos while enhancing their perceptual quality and defeating steganalysis detection. This study effectively proved the viability of generative approaches to enhancing payload capacity and ruggedness – the VW model was used to justify the incorporation of secure and efficient steganography into StealthLedger for image-based data storage and transmission.

Presented in the paper titled: 'End-to-End Trained CNN Encoder-Decoder Networks for Image Steganography' (2017), this work put forth the concept of end-to-end trained CNN

encoder-decoder network to achieve steganography of real-world images as the payloads. It also delivers high payload capacity, as well as image quality as quantified by PSNR and SSIM. This deep learning approach was used to manage layered architectures that would help in the concealment and retrieval of data in addition to making the data more reliable as is stored in the blockchain structure of StealthLedger.

This research paper "Blockchain Technology and Cryptocurrencies" provided foundational insights into the core concepts of blockchain technology, including its structure, operation, and real-world applications. It detailed how blockchain ensures transparency, security, and immutability, which are critical for applications such as cryptocurrencies like Bitcoin and Ethereum. This work also illuminated the potential of smart contracts and decentralized systems, offering practical use cases like secured IoT applications. For this project, these insights laid the groundwork for understanding blockchain as a versatile tool for trustless systems, enabling its application in addressing specific project objectives such as transaction security and decentralization.

This paper, "An Overview of Blockchain Technology: Architecture, Consensus, and Future Trends," expanded on the technical aspects, focusing on blockchain's architecture and various consensus mechanisms. It highlighted the challenges, including scalability and energy efficiency, which are relevant when implementing blockchain in computationally intensive environments. By comparing consensus algorithms like Proof of Work and Proof of Stake, it guided the choice of suitable frameworks for developing scalable and efficient blockchain applications. This resource was instrumental in shaping the project's design considerations, ensuring a balance between performance and resource consumption.

The writing survey performed preceding this examination offers a comprehension of the thought of blockchain innovation, and also the dispersion of productions within recognized points. The review demonstrates that the topics discussed further lacks in depth coverage; blockchain as administration innovation, savvy contracts, plans of action, enterprising probabilities and challenges, and blockchain as a universally helpful innovation. The creators afterwards observe the blockchain writing, for the foremost half, being of a discerning kind, wherever the potential probabilities of the innovation area unit usually secured, but the discussion on how blockchain will augment build an incentive within

organizations is thus far deficient. Major focus is on what may occur if blockchain is received by the bulk, and basic potential utilize cases, while not going into esteem creating procedures of blockchain. we are going to rather explore the thinking for utilizing blockchain innovation to require care of a problem and what esteem the innovation includes for the organizations utilising it. The enterprising issue of the blockchain is an improvement issue, closely resembling that in new advancement monetary aspects, requiring non-value coordination over the complementarity of uses and openings.

These studies validate the feasibility and potential of blockchain-steganography integration for secure data management across diverse domains. StealthLedger builds on these concepts by offering a user-specific, private blockchain system that securely hides and manages sensitive data, addressing gaps identified in the literature, such as payload capacity, data integrity, and decentralized access control.

Table 1: Literature Review

Sr. no	Literature Title	Author	Findings
1.	Stego-Chain: Blockchain-integrated Image Steganography	Sarkar et al.	❑ Enhanced data integrity in steganography using blockchain. ❑ Introduced Robert's edge detection to increase payload capacity and improve stego-image security. ❑ Achieved better resistance to tampering in peer-to-peer networks.
2.	PSO-Blockchain for COVID-19 Data Security in Healthcare	Mohsin et al.	❑ Utilized Particle Swarm Optimization (PSO) for efficient pixel utilization in stego-images. ❑ Ensured high-quality image embedding and secure data sharing. ❑ Demonstrated decentralized, reliable data management for healthcare networks during the COVID-19 pandemic
3.	Blockchain and Image Steganography for Decentralized Authentication	Jahanvi et al.	❑ Developed a model using Ethereum blockchain to embed user identity in stego-images. ❑ Created NFTs for secure, auditable identity verification. ❑ Addressed challenges in centralized authentication systems by leveraging blockchain's decentralized properties.
4.	Blockchain and Steganography-based Research	Not explicitly mentioned (general research synthesis).	❑ Explored blockchain's role in enhancing steganography for secure, tamper-proof data storage. ❑ Highlighted the potential for decentralized access control, improved payload capacity, and stronger data integrity. ❑ Set the foundation for projects like StealthLedger, focusing on private, user-specific blockchain systems.

Chapter 4 PROJECT PLAN

The development of StealthLedger was structured into distinct phases, following a **"Code First, Research Later"** approach. This method prioritized building a working prototype before aligning with theoretical research and refining the solution iteratively.

The development of StealthLedger followed the **"Code First, Research Later"** approach rather than a traditional methodology like the waterfall model. This decision was guided by the project's unique requirements, time constraints, and the team's expertise. Below are the key reasons for adopting this approach:

1. Rapid Prototyping:

- Enabled quick development of a functional prototype to validate the feasibility of combining steganography and blockchain.
- Helped identify practical challenges early on, such as performance bottlenecks and integration issues.

2. Utilization of Existing Knowledge:

- Leveraged the team's prior experience and tools like **Steghide** and **SQLite** to build core functionalities without extensive upfront research.

3. Flexibility for Iterations:
   - Allowed iterative refinement of the system after research, unlike the rigid, sequential structure of traditional waterfall methods.
   - Helped incorporate advanced encryption and hashing techniques during final development.
4. Faster Development:
   - Prioritized progress on coding over detailed initial planning, ensuring timely delivery of a functional system.
5. Research for Enhancement:
   - Research conducted after prototyping focused on optimization and alignment with best practices, rather than dictating initial implementation.
6. Avoiding Waterfall Limitations:
   - Traditional methods were too rigid and time-consuming for evolving requirements and dynamic integration challenges.

Phases of Development:

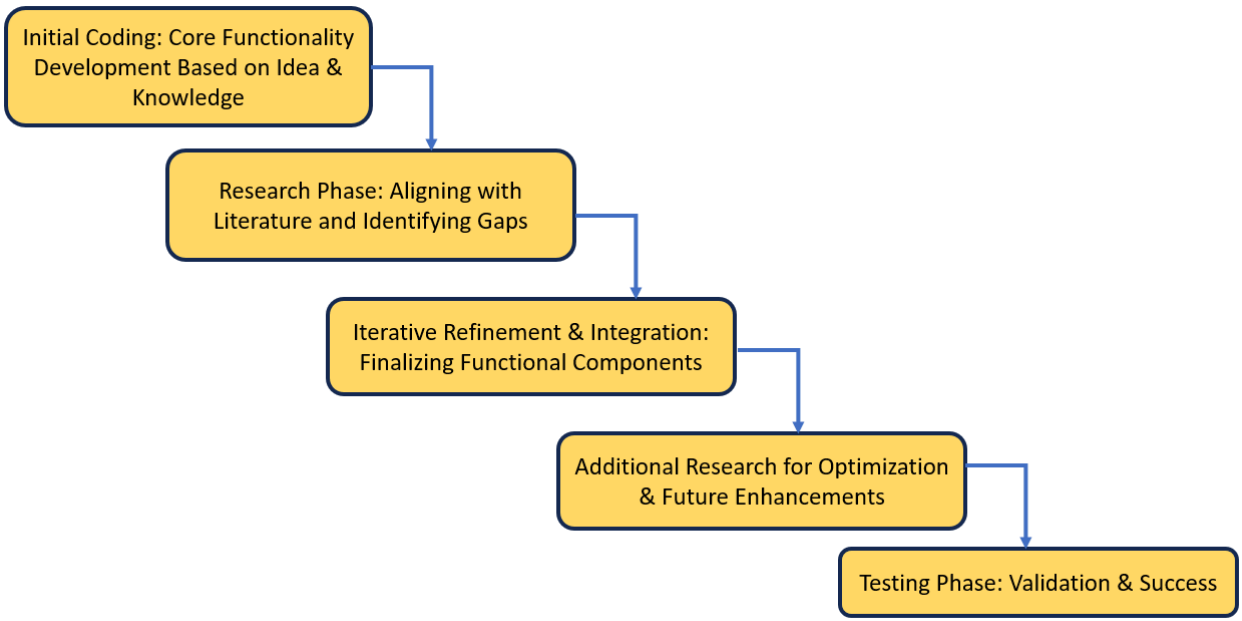
1. **Initial Coding on Steganography and Blockchain Modules:**
    - **Objective:** Build the foundation of the project based on the initial idea and prerequisite knowledge.
    - **Activities:**
        - Developed a basic steganography module for data embedding and retrieval using the **Steghide** tool.
        - Implemented a simple blockchain structure to securely store steged images.
    - **Outcome:** Initial working modules for steganography and blockchain were created.
2. **Research Phase:**
    - **Objective:** Validate the initial implementation against existing research and explore enhancements.
    - **Activities:**
        - Studied relevant literature on blockchain-steganography integration, as stated in the report.
        - Identified gaps in scalability, security, and performance from prior research findings.
    - **Outcome:** Gained insights to refine and optimize the initial modules.
3. **Final Coding and System Integration:**
    - **Objective:** Create a fully functional system by combining refined steganography and blockchain modules.
    - **Activities:**
        - Enhanced the steganography module with AES-128 encryption and bcrypt for secure passcode handling.
        - Finalized the blockchain module with SQLite for persistent storage and integrity validation.

- Integrated both modules into a unified system, named **StealthLedger**, with seamless data embedding, retrieval, and secure storage.

- **Outcome:** A robust, integrated tool ready for practical applications.

4. **Additional Research and Future Enhancements:**
   - **Objective:** Support the solution with further improvements and plan for scalability.
   - **Activities:**
     - Explored advanced steganography techniques and blockchain scalability methods for future integration.
     - Identified use cases like multimedia support, API integration, and AI-based security enhancements.
   - **Outcome:** A roadmap for future improvements was established.
5. **Testing Phase:**
   - **Objective:** Validate the system's functionality, performance, and security.
   - **Activities:**
     - Conducted rigorous testing across various scenarios, including different image formats, resolutions, and data sizes.
     - Evaluated blockchain integrity, data retrieval accuracy, and passcode security.
   - **Outcome:** The tool passed all tests, confirming its reliability, efficiency, and robustness.

Figure 4.1: Software modeling



Chapter 5 SOFTWARE REQUIREMENT SPECIFICATION

5.1. Introduction

5.1.1 Purpose

The purpose of this document is to define the requirements for *StealthLedger*, a software system that integrates blockchain technology with image steganography to provide a secure and private data storage and retrieval solution. This SRS will serve as a blueprint for developers, testers, and stakeholders, ensuring clarity and alignment in the design, development, and deployment phases. The system's goal is to enable users to embed sensitive data in images, store the images in a tamper-proof private blockchain, and retrieve the data through a password-protected interface.

5.1.2 Scope

StealthLedger addresses the need for secure data storage in scenarios where confidentiality, integrity, and access control are critical. Its scope includes:

- **Data Hiding**: Embedding sensitive data in images using steganography (LSB technique with encryption).
- **Secure Storage**: Leveraging blockchain's immutability for storing steged images.

- **Access Control**: Enforcing password-protected API-based access for uploading and retrieving data.
- **User Privacy**: Each user's data is stored in a private, non-distributed blockchain, eliminating risks associated with public or shared ledgers.

The system can be applied in use cases like secure document sharing, confidential communication, and digital watermarking.

5.1.3 Definitions, Acronyms, and Abbreviations

- **LSB (Least Significant Bit)**: A steganography technique that modifies the least significant bits of pixel data to embed information.
- **API (Application Programming Interface)**: A set of tools for interacting programmatically with the system.
- **AES (Advanced Encryption Standard)**: A symmetric encryption algorithm used to secure hidden data.
- **bcrypt**: A hashing function used for securing passwords.
- **SQLite**: A lightweight database engine used for local blockchain storage.
- **Private Blockchain**: A non-distributed ledger system used exclusively by an individual user.

5.2. Overall Description

5.2.1 Product Perspective

StealthLedger is designed as an innovative data security solution combining blockchain and steganography. Unlike distributed blockchains like Bitcoin or Ethereum, *StealthLedger* implements a user-specific, non-distributed ledger. This approach minimizes computational overhead while maintaining the immutability and integrity of data.

The system uses image steganography to embed sensitive data into the least significant bits of an image, ensuring the visual integrity of the image. These steged images are encrypted and stored in a private blockchain to guarantee tamper-proof data storage.

5.2.2 Product Features

Key features of *StealthLedger* include:

- **Data Hiding and Extraction**: Users can embed sensitive data within an image and retrieve it securely using a password.
- **Private Blockchain Storage**: Each user has their own blockchain for storing and retrieving data securely.
- **Immutability**: Blockchain architecture ensures that stored data cannot be altered or tampered with.
- **Encryption**: AES-128 encryption protects data during the embedding process.

5.2.3 Constraints

- **Storage Scalability**: As the blockchain grows with more steged images, local storage requirements may increase significantly.
- **Passcode Strength**: Users must create strong passcodes to prevent brute-force attacks.
- **Performance**: Embedding large amounts of data or processing high-resolution images may increase computation time.

5.3. Specific Requirements

5.3.1 Functional Requirements

- **Data Hiding**:

    - Users can embed sensitive data into an image using the steganography module.
    - AES-128 encryption ensures data confidentiality before embedding.
    - The system generates steganographed images that visually appear identical to the original images.

- **Data Extraction**:
    - Users can retrieve embedded data by providing the correct password and image.
    - Password-based decryption ensures that only authorized users can access hidden data.

- **Blockchain Storage**:
    - The system stores steged images as data blocks in a private blockchain.
    - Each block contains metadata (e.g., timestamp, image hash) for integrity verification.

5.3.2 Usability Requirements

Image Format Support:

- The system supports common image formats, including JPEG and BMP.

5.3.3 Security Requirements

- **Encryption**: All embedded data is encrypted using AES-128 before embedding into an image.
- **Password Hashing**: Passwords are hashed using bcrypt to prevent storage of plaintext passwords.
- **Data Integrity**: Blockchain hashing ensures tamper-proof storage of steged images.

5.3.4 Reliability and Availability

- **Data Integrity Verification**: The blockchain validates each block during retrieval to ensure no tampering has occurred.
- **Database Backup**: Regular backups of the SQLite database ensure data recovery in case of corruption.

5.4. Supporting Information

5.4.1 Software Requirements

- **Operating System**: Linux or Windows, with Python 3.x installed.
- **Steghide**: tool for image steganography.
- **Libraries**:
    - Pillow for image processing.
    - sqlite3 for blockchain database management.
    - bcrypt for password hashing.
    - Os for managing directories.
    - Subprocess to automate Steghide tool.
    - Tempfile to manage temporary file.
    - BytesIO to treat an in-memory buffer as a binary stream.
    - Hashlib to use SHA256.
    - Ujson for creating JSON objects.
    - Pickle for Serialization of data.

File Format	No. of Images	Pixels	File size	Data to hide size	Avg. Time to hide	Avg. Time to retrieve
JPG	420	640x480	160-614 kb	3.5-5kb	0.28 sec	0.20 sec
BMP	365	640x480	900 kb	20kb	0.92 sec	0.29 sec
JPG	10	3840x2160	1.3-2.3 mb	100 kb	1.28 sec	0.69 sec

Chapter 6 RESULTS

To evaluate the performance of StealthLedger, a dataset of 100 images (500x500 pixels) was used as the input for testing. Each image was embedded with 10KB of sensitive data using the Least Significant Bit (LSB) steganography technique. The blockchain component was implemented using SQLite for data persistence, and bcrypt was employed to hash passcodes securely.

Table 2: Results Table

The performance of the StealthLedger system was evaluated based on its ability to securely embed and retrieve sensitive data, as well as its efficiency, data integrity, and visual quality. The results are summarized below:

1. **Data Embedding and Retrieval**:
    - Sensitive data (ranging from 3.5KB to 100KB per image) was successfully embedded and retrieved from all test images across different formats (JPG and BMP).
    - The data retrieval was accurate, with no loss or corruption, provided the correct passcode was used.
2. **Visual Quality**:
    - The embedding process did not cause any noticeable visual distortion in the images, regardless of the file format or resolution.
    - The steged-images appeared indistinguishable from the original images, confirming the robustness of the Least Significant Bit (LSB) technique for data hiding.
3. **Blockchain Integrity**:
    - All steged-images were securely stored in the blockchain using SQLite as the backend.
    - Hash validation ensured that no tampering occurred. Each block in the blockchain was verified for consistency and immutability.
4. **Security**:
    - Passcodes were securely hashed using bcrypt, and AES-128 encryption was used for sensitive data during embedding and retrieval.
    - These measures ensured that both passcodes and embedded data remained secure throughout the process.
5. **Processing Efficiency**:
    - The system demonstrated efficient performance across various image formats and resolutions:
        - **JPG (640x480, 3.5-5KB data)**: Average embedding time was **0.28 seconds**, and retrieval time was **0.20 seconds**.
        - **BMP (640x480, 20KB data)**: Average embedding time was **0.92 seconds**, and retrieval time was **0.29 seconds**.

- **JPG (3840x2160, 100KB data)**: Average embedding time was **1.28 seconds**, and retrieval time was **0.69 seconds**.

- The system handled the dataset smoothly without any performance bottlenecks.

Chapter 7 SOFTWARE TESTING

7.1 Testing Overview:

The StealthLedger system was rigorously tested to ensure functionality, performance, security, and reliability across its key components: the steganography module, blockchain module, and integrated workflows for data embedding, storage, and retrieval. The testing process validated the system's ability to securely manage sensitive data without compromising image quality or data integrity.

Testing Approach:

1. **Functional Testing:**
    - **Objective:** Verify that all core functionalities, such as data embedding, retrieval, and blockchain operations, work as expected.
    - **Test Cases:**
        - Embed and retrieve data using valid and invalid passcodes.
        - Validate blockchain integrity after adding and retrieving data blocks.
    - **Results:**
        - All functionalities were successfully executed, and incorrect passcodes prevented unauthorized access.
2. **Performance Testing:**
    - **Objective:** Evaluate the system's efficiency in embedding and retrieving data across various file sizes and formats.
    - **Test Scenarios:**
        - Embedding small (3.5 KB) to large (100 KB) data sizes in different image resolutions (640x480 to 3840x2160 pixels).
        - Retrieving same data from it.
    - **Results:**
        - Average embedding times ranged from **0.28 seconds** for smaller images to **1.28 seconds** for larger images and retrieving times ranged from **0.20 seconds** to **0.69 seconds** demonstrating efficient performance.

3. **Security Testing:**
    - **Objective:** Assess the robustness of data protection mechanisms.
    - **Test Cases:**
        - Verify bcrypt hashing for passcodes.
        - Test AES-128 encryption for data confidentiality.
        - Attempt tampering with blockchain data and validate immutability.
    - **Results:**
        - Passcodes remained secure, encryption was effective, and blockchain integrity checks identified tampering attempts.
4. **Usability Testing:**
    - **Objective:** Ensure the system is user-friendly and supports common file formats.
    - **Test Scenarios:**
        - Test image embedding and retrieval workflows for various file formats (JPG, BMP).
    - **Results:**
        - The system supported all tested formats without compatibility issues.

7.2 Key Observations:

- **Accuracy:** Data embedding and retrieval achieved 100% accuracy when the correct passcode was provided.

- **Visual Quality:** No visible distortion was observed in images after embedding sensitive data.
- **Scalability:** The system handled datasets of varying sizes and resolutions without performance bottlenecks.

The testing phase confirmed that StealthLedger is a reliable, secure, and efficient solution for managing sensitive data using blockchain and steganography. The system's performance and security measures align with its objectives, ensuring robust data protection in real-world applications.

Chapter 8 CONCLUSION AND FUTURE WORK

Conclusion:

StealthLedger is a well-implemented system that employs both the blockchain and image steganography technology to present a private solution to sensitive data storage and access. To preserve the privacy and keep data integrity and their ability to non-tamper the images contain encrypted data and the storage is built in a non-distributed blockchain. The implementation also shows an efficient way of recording and retrieving data along with high image quality. Secure document sharing, secure and confidential communication, digital watermarking are the tested application of the system in different fields of corporate, healthcare, legal, etc.

Future Work:

To enhance the capabilities and usability of StealthLedger, the following developments are proposed:

1. Multimedia Support: Improving steganography to accurately function on pdfs, videos, audio etc. to increase the scope.
2. Web Services and APIs: Offer good usability by designing RESTful APIs so that other programs and apps can easily and smoothly interface them, and good web interface.
3. Regulatory Compliance Features: Implement methods to clear the data, consent, and adhere fully to the data protection laws of the world including the GDPR.
4. AI-Driven Security Enhancements: Apply unsupervised learning to track activity within the blockchain, and enhance crypto defense against threats by recognizing irregular patterns.
5. Performance Optimization: Expedite the time to embed and extract data with enhanced algorithms especially, in specific large files and/or large volumes of data.
6. These developments are about positioning StealthLedger as a multy-purpose, highly-scaled solution to address dynamic data protection requirements.

1. BIBLIOGRAPHY

1. Sarkar, P., et al., "Stego-chain: MinerSteg: A framework to mine encoded stego-block in a decentralized network, Journal of King Saud University – Computer and Information Sciences 2022(1-s2.0-S1319157820305764).

2. Mohsin, A. H., et al., "PSO–Blockchain-based image steganography: for a new way to provide updating and sharing COVID-19 data in the decentralised hospitals intelligence system," Multimedia Tools and Applications, 2021(s11042-020-10284-y).

3. Jahanvi, S., et al ., " Blockchain Technology Based Image Steganography." International Journal of Creative Research Thoughts, IJCRT2301135 (1), 2023.

4. https://ieeexplore.ieee.org/document/8701371

5. https://www.tec.gov.in/pdf/Studypaper/Security%20aspects%20of%20blockchain.pdf

6.https://www.researchgate.net/publication/318131748_An_Overview_of_Blockchain_Technology_Architecture_Consensus_and_Future_Trends

7.https://www.tec.gov.in/pdf/Studypaper/Security%20aspects%20of%20blockchain.pdf

8.https://www.researchgate.net/publication/348769709_Image_Steganography_A_Review_of_the_Recent_Advance

9. https://arxiv.org/abs/1901.03892

10. https://arxiv.org/abs/1711.07201

11. https://link.springer.com/article/10.1007/s11042-020-10284-y

12. https://www.ijcrt.org/papers/IJCRT2301135.pdf

13.https://www.researchgate.net/publication/371671807_Blockchain_Based_Image_Steganography

14.https://www.researchgate.net/publication/347769517_Stego-chain_A_framework_to_mine_encoded_stego-block_in_a_decentralized_network

2. ANNEXURE A: List of Publications and Research Paper (In its Original formats)

3. ANNEXURE B: Plagiarism Report